

Worksheet No. - 03

Student Name: Sukhleen Kaur

UID: 25MCA20031

Branch: MCA

Section/Group: 25MCA-6B

Semester: 2nd

Date of Performance: 17.02.2026

Subject Name: Design and Analysis of Algorithm Lab **Subject Code:** 25CAP-612

Aim/Overview of the practical: To find the Minimum Cost Spanning Tree (MST) of a given undirected weighted graph using Kruskal's Algorithm, by selecting the minimum weight edges without forming any cycle.

Objective: To implement Kruskal's Algorithm for constructing the Minimum Cost Spanning Tree of a given undirected weighted graph, by selecting edges in increasing order of weight while avoiding cycles using the Disjoint Set Union (DSU) method.

Procedure/Algorithm/Code

Algorithm

1. Read all vertices and edges.
2. Sort edges in increasing order of weight.
3. Initialize DSU (Disjoint Set).
4. Pick the smallest edge one by one.
5. Add edge if it does not form a cycle.
6. Repeat until MST has $(V-1)$ edges.
7. Print MST edges and minimum cost.

Code:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class DSU {
```

```
    vector<int> parent, rank;
```

```
public:
```

```
    DSU(int n) {
```

```
parent.resize(n);

rank.resize(n, 0);

for (int i = 0; i < n; i++)
    parent[i] = i;
}

int find(int node) {
    if (parent[node] == node)
        return node;

    return parent[node] = find(parent[node]);
}

void unite(int u, int v) {
    u = find(u);
    v = find(v);

    if (u != v) {
        if (rank[u] < rank[v])
            parent[u] = v;

        else if (rank[u] > rank[v])
            parent[v] = u;

        else {
            parent[v] = u;
            rank[u]++;
        }
    }
}

};
```

```
struct Edge {  
    int u, v, weight;  
};  
  
bool compare(Edge a, Edge b) {  
    return a.weight < b.weight;  
}  
  
int main() {  
    int V, E;  
  
    cout << "Enter number of vertices and edges: ";  
  
    cin >> V >> E;  
  
    vector<Edge> edges(E);  
  
    cout << "Enter edges (u v weight):\n";  
  
    for (int i = 0; i < E; i++) {  
        cin >> edges[i].u >> edges[i].v >> edges[i].weight;  
    }  
  
    sort(edges.begin(), edges.end(), compare);  
  
    DSU dsu(V);  
  
    int mstCost = 0;  
  
    vector<Edge> mstEdges;  
  
    for (auto edge : edges) {  
        if (dsu.find(edge.u) != dsu.find(edge.v)) {  
            dsu.unite(edge.u, edge.v);  
            mstCost += edge.weight;  
            mstEdges.push_back(edge);  
        }  
    }
```

```
}  
  
cout << "\nEdges in Minimum Spanning Tree:\n";  
  
for (auto e : mstEdges) {  
    cout << e.u << " - " << e.v << " (Weight: " << e.weight << ")\n";  
}  
  
cout << "\nMinimum Cost of MST = " << mstCost << endl;  
  
return 0;  
  
}
```

Output

```
Enter number of vertices and edges: 4 5  
Enter edges (u v weight):  
0 1 10  
0 2 7  
0 3 6  
1 4 11  
2 4 8  
  
Edges in Minimum Spanning Tree:  
0 - 3 (Weight: 6)  
0 - 2 (Weight: 7)  
0 - 1 (Weight: 10)  
  
Minimum Cost of MST = 23
```

Learning outcomes (What I have learnt):

1. Understand the concept of Minimum Spanning Tree (MST).
2. Learn how Kruskal's Algorithm works for finding MST.
3. Apply sorting of edges based on weights.
4. Calculate the minimum cost required to connect all vertices.